BÁO CÁO CÔNG VIỆC HÀNG NGÀY (DAILY REPORT)

Dành cho Sinh viên Thực tập / Học việc - Vị trí: Lập trình Giao diện (Frontend Developer)

• Họ và tên sinh viên: Đoàn Đức Hiếu

• Mã số sinh viên : 6751071022 | Trường: trường Đại Học Giao Thông Vận Tải chi nhánh 2 (UTC2)

• Người hướng dẫn (Mentor): Th.s Trần Thị Dung

• Ngày báo cáo: từ ngày 18/10/2026 đến ngày 25/10/2026

• Dự án : hệ thống đố vui trắc nghiệm quizz

I. CÔNG VIỆC ĐÃ THỰC HIỆN TRONG NGÀY

+ Ngày 18/10/2026 : Nhận phần code chính từ bạn Bùi Việt Hoàng , bắt tay vào làm giao diện in phông chữ để ra kết quả đẹp mắt trình bày cho người xem (nhờ AI phác thảo mô hình giao diện và tối ưu phần chọn đáp án bằng cách di chuột là chọn được)

+ Ngày 19/10/2026 -22/10/2026 : tìm hiểu cách trình bày giao diện nếu phần câu trả lời hoặc câu hỏi bị dài quá vượt ra khung hiển thị thì làm như nào , tự mình nghĩ ra ý tưởng xem phần nội dung hiện thị sẽ chia ra làm ba cái rongnoidung sẽ cố định cứ mỗi hàng sẽ có 98 kí tự cố dịnh cho mỗi hàng hiển thị , cứ phần đáp án nào có số ký tự dài hơn 98 kía tự thì chương trình sẽ tự động tìm dấu cách gần nhất gần 98 ký tự nhất để làm lệnh in xuống dòng (\n) , đường link dẫn tới phần vẽ đáp án (https://github.com/thanhhaole021108-ctrl/quizzle-system/blob/main/Code_giao_dien.c#L551-L580),

- Hàm dùng để phát hiện dấu cách gần nhất sau 98 kí tự (https://github.com/thanhhaole021108-ctrl/quizzle-system/blob/main/Code_giao_dien.c#L571) ,
- `dai` ban đầu là 98, tức giới hạn ký tự của một dòng.
- `conLai[dai] != ' '` kiểm tra ký tự tại vị trí `dai` có phải dấu cách không.
- Nếu chưa phải dấu cách, `dai--` lùi về trước một ký tự.
- Vòng lặp dừng khi gặp dấu cách hoặc lùi về đầu chuỗi.
- Sau đó `dai` được dùng để chép và in phần đầu. Ví dụ, nếu ký tự thứ 98 nằm giữa hai từ, hàm lùi tới dấu cách trước đó để không tách đôi từ.

+Thêm vào đó là hàm trộn ngẫu nhiên câu hỏi : hàm này nằm ở (https://github.com/thanhhaole021108-ctrl/quizzle-system/blob/main/Code_giao_dien.c#L1024-L1033)

- +Tự mình tìm hiểu hàm mới là rand() dùng để chọn ngẫu nhiên ví dụ trong ngân hàng câu hỏi thì vị trí của các câu hỏi được xác định , thì hàm này dùng để quét các câu hỏi dùng để đảo vị trí lẫn lộn với nhau
- `rand()` tạo ra một số giả ngẫu nhiên.
- `% (i + 1)` giới hạn số đó vào phạm vi vị trí cần chọn.

+ Còn hàm srand (time(NULL)) dùng để đổi mỗi lần làm một bài kiểm tra mới sẽ là một đoạn xáo trộn khác với lần trước đó đảm bảo thứ tự xắp xếp ngẫu nhiên của một câu là hoàn toàn khác nhau không liên quan với lần làm bài kiểm tra trước đó

- `srand(time(NULL))` đặt điểm bắt đầu cho dãy số; nhờ vậy, chạy chương trình vào thời điểm khác nhau thường sẽ cho thứ tự khác.

+Hàm trộn đáp án (https://github.com/thanhhaole021108-ctrl/quizzle-system/blob/main/Code_giao_dien.c#L1035-L1052) cũng tương tự với hàm trộn câu hỏi đều sử dụng rand() để trộn câu hỏi lại với nhau
+Phần tính điểm (https://github.com/thanhhaole021108-ctrl/quizzle-system/blob/main/Code_giao_dien.c#L1369-L1373)

+) `soCauDung` tăng mỗi khi trả lời đúng. Ép sang `float` để phép chia có thể cho kết quả thập phân thay vì chỉ lấy phần nguyên.

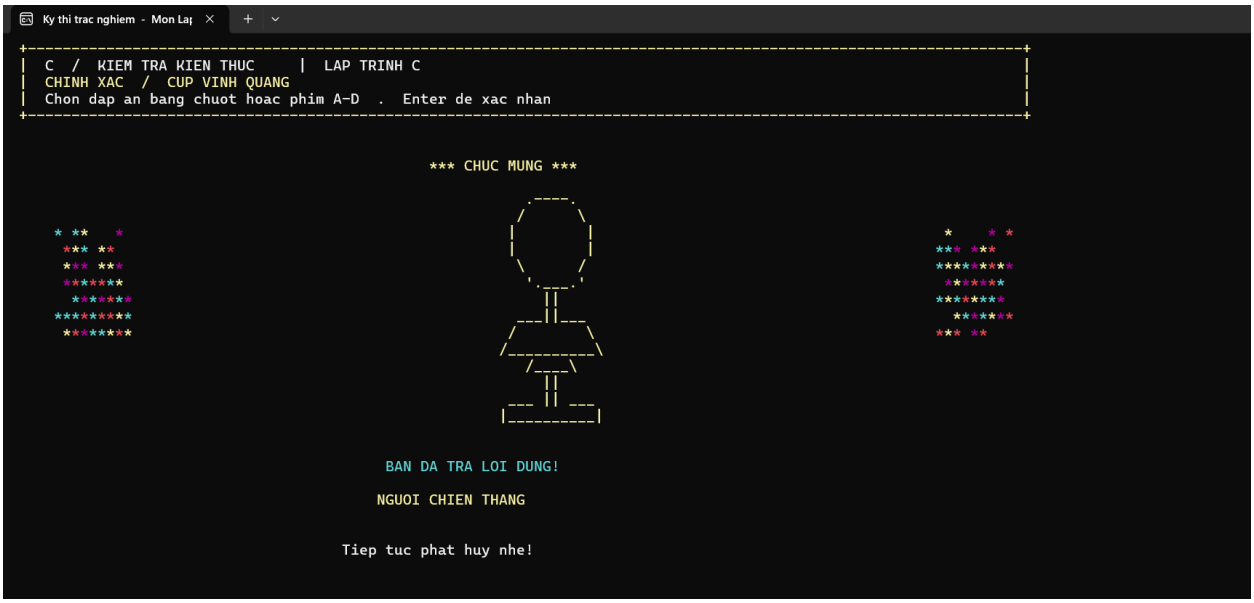
Ví dụ: làm 20 câu đúng 12 câu:

- Phần trăm: `12 / 20 × 100 = 60%`
- Điểm: `60 / 100 × 10 = 6.0 / 10`

Cách này tương đương lấy số câu đúng chia tổng số câu rồi nhân 10.

Trong đó nếu tổng điểm cuối cùng trên 4 hoặc bằng 4 sẽ in đạt , còn nếu dưới 4 sẽ in chưa đạt .

+22/10/2026-25/10/226: kêu AI chỉnh sửa giao diện cho đẹp , sử dụng các hàm truyền biến cho nhau như struct , void , int , đặc biệt sử các hàm printf để in ra các ký tự khi bạn trả lời sai , đúng tương ứng với các hoạt ảnh khác nhau ví dụ như :



Hình 1: xuất hiện khi bạn trả lời đúng



Hình 2 : Xuất hiện khi bạn trả lời sai và xuất hiện lại đáp án đúng



Hình 3: xuất hiện khi bạn trả lời đúng , xuất hiện ngẫu nhiên với hình 1 để tránh lặp lại tạo sự nhàm chán

+) Tất cả các hình trên đều dùng hàm printf để in ra màn hình được cho vào hàm riêng biệt để khi bạn trả lời đúng thì chương trình tự biết gọi các hàm nào ra để chúc mừng .

+Dưới đây là các hàm tự tạo ra để sử dụng trong chương trình này :

Khởi tạo và điều khiển giao diện console

- `khoiTaoConsole()` — lấy handle console, đặt tiêu đề và kích thước cửa sổ, bật chế độ chuột.
- `choConsoleDuKichThuoc()` — chờ người dùng phóng to cửa sổ nếu console quá nhỏ để vẽ giao diện.
- `batCheDoChuot()` — bật chế độ nhận sự kiện chuột.
- `anConTro()` — ẩn con trỏ nhập.
- `hienConTro()` — hiện con trỏ nhập.
- `toiViTri(x, y)` — đưa con trỏ console đến tọa độ `(x, y)`.
- `mauNenToi(mau)` — chuyển mã màu phù hợp với nền tối.
- `caiDatMau(mau)` — đặt màu chữ và nền cho nội dung sắp in.
- `toVung(x, y, w, mau)` — tô màu một vùng console.
- `toNen(mauChuNen)` — tô nền toàn bộ màn hình console.
- `xoaManHinh()` — xóa nội dung màn hình và đặt lại màu nền.

In chữ, câu hỏi và khung

- `inTai(x, y, mau, s)` — in chuỗi `s` tại tọa độ `(x, y)` với màu được chọn.
- `inDong(mau, s)` — in một dòng chữ với màu được chọn.
- `veKhung(x, y, w, h, mau)` — vẽ khung có vị trí, chiều rộng, chiều cao và màu chỉ định.
- `veBannerDau(phuDe)` — vẽ banner đầu trang và hiển thị tiêu đề phụ.
- `inCauHoiXuongDong(x, y, mau, s)` — chia câu hỏi thường thành các dòng để vừa vùng hiển thị.
- `inCauHoiNguon(x, y, mau, s, gioiHanDong)` — hiển thị câu hỏi có mã hoặc nội dung nhiều dòng.
- `veTheDapAn(y, kyTu, nd, dangHover)` — vẽ thẻ đáp án dạng gọn.
- `veTheDapAnDayDu(y, kyTu, nd, dangHover)` — vẽ thẻ đáp án đầy đủ, có ngắt nội dung dài thành dòng.
- `veHopXemTruoc(y, nhan, noiDung, dangChon)` — vẽ khung xem trước đáp án khi rê chuột.
- `veThanhTienDo(x, y, rong, hienTai, tong)` — vẽ thanh tiến độ.
- `veRongKhung(khung)` — vẽ hiệu ứng rồng bằng ký tự trong console.

Âm thanh và hiệu ứng

- `phatAmThanh(loai)` — phát âm thanh tương ứng với đúng, sai hoặc thao tác bấm.
- `hienThongBaoLapLanh(noiDung, x, y, soNhip)` — làm thông báo nhấp nháy.
- `hieuUngCupXoay()` — chạy hiệu ứng chiếc cúp xoay.
- `hieuUngRongPhunLua()` — chạy hiệu ứng rồng/phun lửa.
- `hieuUngPhaoHoaBenLe()` — chạy hiệu ứng pháo hoa hai bên màn hình.
- `inNoiDungDapAnSai(x, y, noiDung)` — ngắt và in nội dung đáp án đúng khi thí sinh trả lời sai.
- `hieuUngSai(dapAnDung, noiDungDapAnDung)` — hiển thị hiệu ứng và đáp án đúng khi trả lời sai.
- `hieuUngHetGio(dapAnDung, noiDungDapAnDung)` — hiển thị hiệu ứng khi hết thời gian.

Nhập liệu, chọn đáp án và kết quả

- `docThongTin(chuoi, kichThuoc, x, y, rong)` — đọc thông tin bằng bàn phím và hiển thị trong vùng nhập cố định.

- `nhapThongTinNguoiThi(thiSinh)` — vẽ màn hình và nhập họ tên, mã số sinh viên.
- `chonDapAnBangChuot(...)` — hiển thị câu hỏi, nhận lựa chọn bằng chuột hoặc phím, cập nhật đáp án đang rê tới và theo dõi thời gian.
- `choClickThe(soThe, yThe, hThe, x1, x2)` — xử lý rê chuột và bấm chọn một trong các thẻ.
- `chonMenuBangChuot()` — hiển thị menu chọn phần thi.
- `hoiTiepTucBangChuot(...)` — hiện bảng kết quả cuối bài và cho chọn làm bài mới hoặc kết thúc.

Trộn đề và điều phối chương trình

- `tronDeThi(ds, soLuong)` — trộn thứ tự câu hỏi trong mảng bằng cách đổi chỗ các câu.
- `tronDapAn(ch)` — trộn bốn lựa chọn của một câu và cập nhật vị trí đáp án đúng.
- `main()` — khởi tạo chương trình, chép ngân hàng câu hỏi vào đề, gọi các hàm trộn, chạy bài thi và tính điểm/phần trăm.

II. KIẾN THỨC MỚI HỌC ĐƯỢC / KINH NGHIỆM ĐÚC KẾT

- Biết sử dụng và làm quen các hàm struct để gọi ra đề thi , không cần phải gõ đi gõ lại thay vào đó chỉ gọi và biết sử dụng các hàm khác mình chỉ cần gõ một lần và bất cứ khi nào thích có thể gọi ra để chạy chương trình đó khi cần

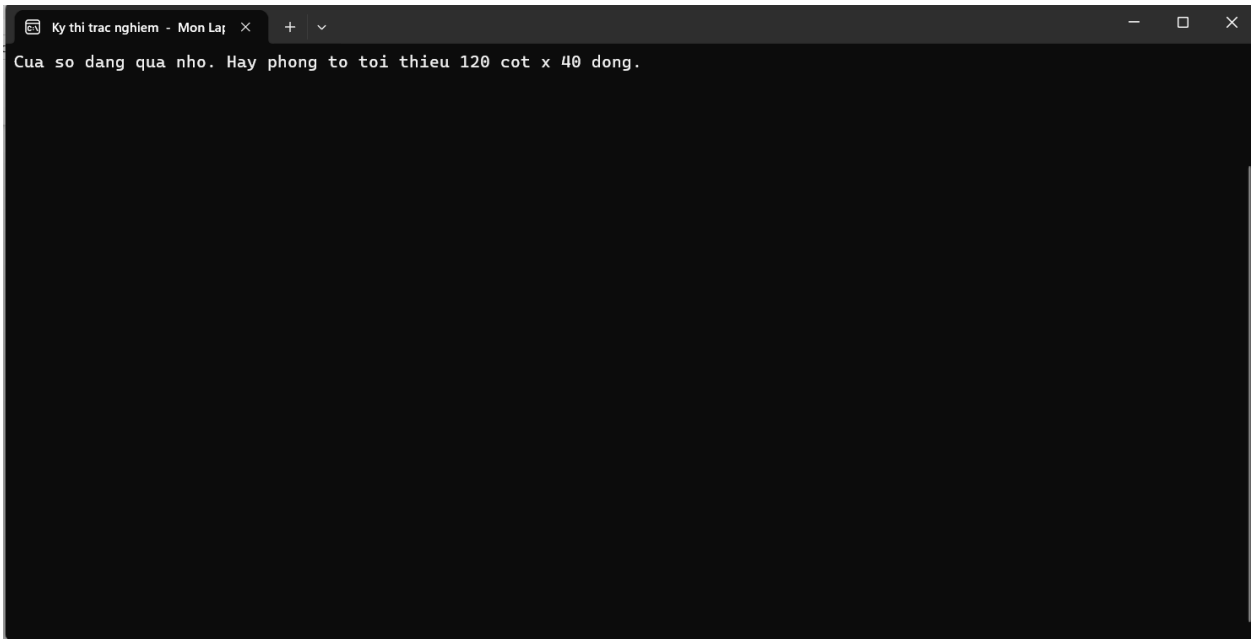
- Học cách quản lí dự án làm theo trình tự như nào hiểu được cách tạo một chương trình chạy được ra làm sao

- Rút kinh nghiệm: Khi mình printf mà dòng chữ mình cần in quá dài sẽ làm mất thẩm mĩ và làm cấu trúc bài thi không được đẹp mắt và phù hợp thẩm mĩ mắt người xem thì phải biết cần khi nào phải xuống dòng và xuống dòng như nào để không bị lỗi cắt chữ ngang

III. KHÓ KHĂN / VƯỚNG MẮC KHI LÀM BÀI :

- Vấn đề gặp phải: Giao diện bị lỗi vì không biết khi bật chương trình thì sẽ hiển thị ra khung hình nhỏ sẽ bị lỗi phông chữ vì khi in ra màn hình mình đã ngầm xác định là phải in ra chỗ đó mà khi bật tắt thu nhỏ màn hình sẽ vô tình làm không gian hiển thị vô tình bị thu nhỏ đi dẫn đến việc mất chữ cần in ra hoặc mất luôn đoạn in đó

- Đề xuất hướng giải quyết: Em đang tra cứu và nghĩ ra cách là khi màn hình bị thu nhỏ ảnh hưởng đến phần hiển thị chương trình sẽ biết và đóng phần giao diện câu hỏi và yêu cầu người dùng cần phóng to màn hình hiển thi để tiếp tục chạy chương trình. Ví dụ:



Hình 4:Xuất hiện khi màn hình quá nhỏ ảnh hưởng đến việc hiển thị

+Và sau khi người dùng bấm phóng to ra màn hình câu hỏi sẽ trả về nguyên vẹn ban đầu